

Exercise Sheet 2: From C to Assembly

Remarks: These exercises are your homework, and the solutions will be discussed at the exercise on Monday, Nov 7. Although it is not necessary to participate in the exercise or hand in your solutions, contents of the exercise will be relevant for the exam.

Exercise 1:

Calling Conventions: We continue to work on the programs `myprog.c` and `mylib.c` from last exercise sheet.

- (a) By default, `gcc` compiles functions on x86-64 for Linux targets using the System V ABI calling convention. When we have control over both the caller and callee function, we can change the calling convention arbitrarily as long as both the caller and callee follow the *same* convention.

Change both the declaration of `alg` in `myprog.c` and the definition in `mylib.c` so that they use the Microsoft calling convention:

```
int __attribute__((ms_abi)) alg(int n)
```

- (b) Compile the program again, and spot the differences in the assembly code.

```
$ gcc -o myprog_mscall myprog.c mylib.c -O0 -e my_entry -nostdlib -  
    nostartfiles -lc  
$ objdump -d -M intel myprog_mscall
```

Exercise 2:

Using a debugger: With a debugger, you are able to inspect the process during runtime.

- (a) To make the debugger even more useful, add debugging symbols (`-g`) during compilation. These enable the debugger to match assembly code with source code.

```
$ gcc -o myprog_debug myprog.c mylib.c -O0 -e my_entry -nostdlib -  
    nostartfiles -lc -g
```

- (b) Look up the exact definitions of `call` and `ret` in the Intel Instruction manual which you can find linked at the course's moodle page. The *near* variants (6.4.1) are sufficient. Take a look at Figure 6-1.
- (c) `gdb` is a very powerful tool with many options. We will only cover a few of them here.

```
$ gdb myprog_debug  
  
# Set the disassembly syntax to Intel syntax instead of AT&T syntax  
(gdb) set disassembly-flavor intel  
  
# Run the program, but immediately stop at the first instruction  
(gdb) starti
```

```
# Inspect the memory map of the process
(gdb) info file

# Get information about current stack frame
(gdb) info frame

# Print all registers
(gdb) info registers

# Dump memory at the stack frame
(gdb) x/10x $rsp

# Disassemble at the current address
(gdb) disassemble

# Single-step
(gdb) step
```

Step through the program and watch the registers and stack change. Make sure you understand what a *stack frame* is. Why is it important? How do the *base pointer* and *stack pointer* work together?

Exercise 3:

Writing pure assembly: During the previous exercise sheet, we have compiled a quite minimal version of our program, telling `gcc` not to use any libc-related start up files. Now, we are going one step further: We write pure assembly code and don't use any compiler at all.

Note: This exercise *only* works under Linux x86_64.

- (a) Download the file `hello.s` and install `nasm`.

```
# Ubuntu
$ sudo apt install nasm
# Arch Linux
$ sudo pacman -Syu nasm
```

Assemble the program and link it.

```
$ nasm -f elf64 -o hello.o hello.s
$ ld -o hello hello.o
$ ./hello
```

- (b) Take a look at the sections:

```
$ objdump -x hello
```

`nasm` recognized the canonical section names `.rodata` and `.text` and used appropriate flags.

Change the name of the `.text` section to `.mycodesection`, and see what happens. Look up the definition of a *segmentation fault*.

In order to fix the program, tell `nasm` to set the executable flag:

```
section .mycodesection exec
```

- (c) Currently, the string `msg` is located in the `.rodata` section. In the last part of this exercise, we want to change this string in memory. What would happen if we were to write to this memory location with the current section flags?

Change the section name to `.data` and check whether the flags now allow write access.

- (d) Take a look at the disassembled code. Fortunately, comments are describing what is happening. Follow the code using a system call table¹. Why won't this program run on an operating system different than Linux?

Why does the program from the previous exercise sheet (`myprog.c` and `mylib.c`) work with different operating systems?

- (e) Implement the `_addone` function so that it increments the values at a given memory location. In C syntax, the function would look like this:

```
void _addone(void* memloc, size_t nbytes)
{
    for (size_t i = 0; i < nbytes; ++i)
        memloc[i]++;
}
```

It is up to you how the arguments are passed to the function! One example would be to make the caller push both arguments on the stack right before the call.

Call `_addone` with the arguments `msg` and `msglen`. You can confirm the correct implementation by looking at the output.

Hint: Look at the disassembly from the previous part: See how the assembler replaced the labels for `msg` and `_addone` with actual addresses. You can use such labels (and define your own!) also with jump instructions in order to implement a *goto* which may be useful for a for-loop.

Hint: Useful assembly instructions could be `inc`, `je`, `cmp`, `mov`, `jmp`

Hint: Writing correct assembly code can be very hard because there is no compiler to help you at all (besides syntax errors). Use the debugger, step through your program, watch memory registers and the stack itself to find errors. Similar to `gcc`, `nasm` also supports the option `-g`.

¹https://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/